

adaptFVC 取扱説明書

画像から植被率を測る対話型セグメンテーションツール

Version 3.2.0（2026年11月）

岡山大学　作物生産技術学　特任助教　中嶌洸太

目次

1. はじめに

第1部　使う

2. 使い方
3. 出力と自動保存
4. 結果を信じる前に
5. トラブルシューティング
6. ショートカット一覧

第2部　理解する・記録する

7. 植被率（FVC）とは何か
8. 原理
9. 設計の考え方
10. 再現性
11. 論文などで引用するとき
12. ミニ用語集

参考文献

1. はじめに

adaptFVC は、圃場を撮影した画像から**植被率**を計算する Windows 用のアプリです。ドローンのオルソ画像から切り出した画像でも、地上から真上に向けて撮った写真でも使えます。フォルダ内の数百〜数千枚を一括で処理し、1枚ごとの植被率を Excel に出力します。インターネットへの通信は一切しません。

アプリは植被率を自動で計算しますが、**その数字が正しいかを判断できるのは使う人だけ**です。

この説明書は2部に分かれています。**第1部「使う」**は操作の手順と結果の確かめ方、**第2部「理解する・記録する」**は植被率と判定の原理、設計の考え方、再現性と引用のしかたです。操作するだけなら、第1部を読めば足ります。

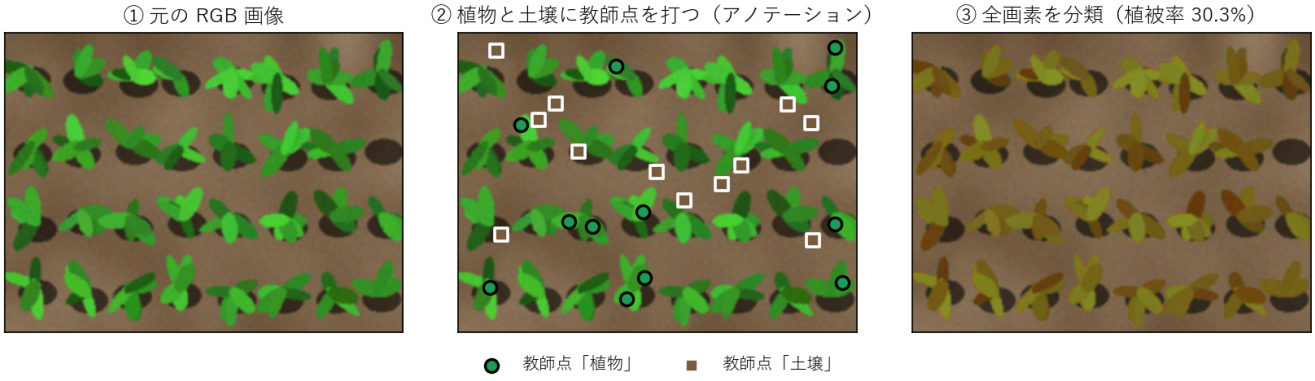


図1｜処理の流れ。① RGB 画像に、② 人が植物と土壌の画素を十数点ずつクリックして教師点を打つと（アノテーション）、③ アプリが残りの全画素を分類し、植被率を計算する。③ では「植物」と判定された画素に赤を半透明で重ねている（緑の葉の上では黄褐色に見える）。

第1部　使う

操作の手順、出力、結果の確かめ方、困ったときの対処。操作するだけなら、第1部を読めば足ります。

最短手順

1. 同一圃場・同一日・同一カメラの画像を、1つのフォルダにまとめます（2.1節）。
2. `adaptFVC.exe` を起動し、そのフォルダを選びます（2.2節）。
3. `Q`（植物）と `W`（土壌）を切り替えながら、画像をクリックして教師点を打ちます（2.3節・2.4節）。
4. `T` で学習し、`A` / `D` で画像を切り替えて判定を確かめます。誤分類の箇所に教師点を足して、もう一度 `T`（2.5節）。
5. プレビューで、学習に用いていない画像でも正しいかを確かめます（2.5節）。
6. 「全画像を処理」を押し、判定画像を目で確かめます（2.6節・第4章）。

2. 使い方

左パネルの3つの欄（「**1 学習データの準備**」→「**2 学習と確認**」→「**3 全画像の処理**」）を上から順に進めます。各欄の**緑の大きなボタン**が、その欄の主な操作です。モデルの読込・保存、画像の追加、出力フォルダを開く、も各欄の小さなボタンにあります。ほとんどの操作は**左手のキー**（第6章）で行え、右手はマウスでクリックに専念できます。

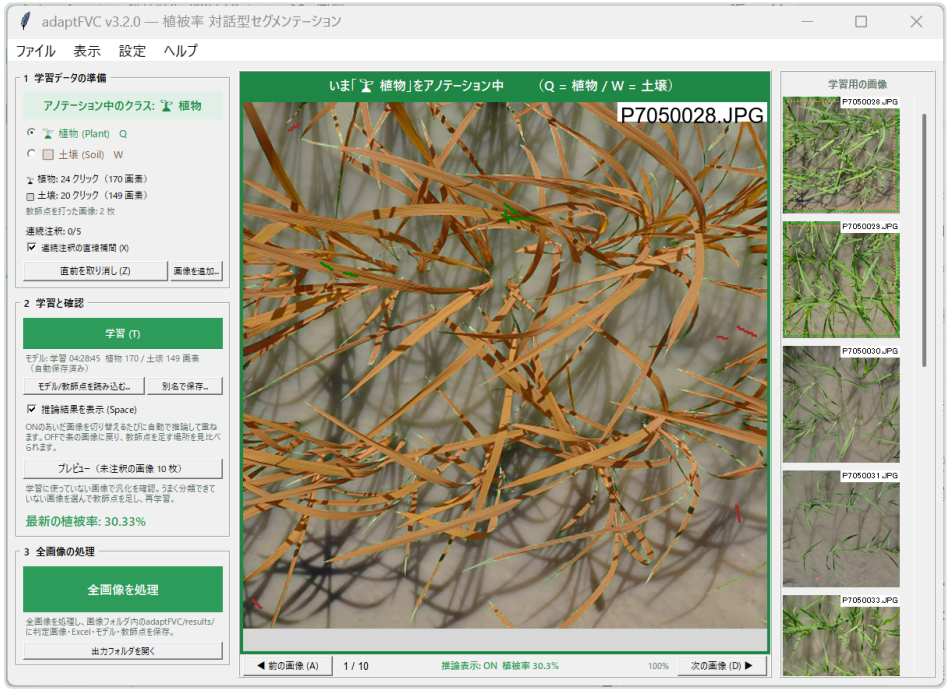


図2｜メイン画面（v3.2.0）。左=3つの欄（左上に、アノテーション中のクラスと、クラスごとのクリック数・画素数）、中央=画像（クリックで教師点を打つ。推論表示が ON のあいだは「植物」と判定された画素に赤が重なる。上の帯と枠の色がアノテーション中のクラス）、下=前後の画像と植被率、右=学習用の画像。

2.1 画像フォルダを用意する（モデルを作る単位）

1つのモデルが通用するのは、色の見え方が揃った画像セットの中だけです。**同一圃場・同一日・同一カメラで撮った画像を1つのフォルダにまとめ、フォルダごとにモデルを作って処理してください。**撮影日・圃場・カメラが変わったら、モデルを使い回さずに学習データを準備し直します。晴れと曇り、午前と夕方が混ざると、どこに教師点を足しても両立しないことがあります。どこでフォルダを分けるべきか分からなければ、プレビュー（2.5節）でどの画像の誤分類が目立つかを確かめると、明るさや撮影の時間帯による違いが見えてきます。対応する画像の形式は JPG・PNG・8bit TIFF です。

2.2 起動と読み込み

1. GitHub のリリースページ（github.com/KotaNakajima/adaptFVC/releases）から `adaptFVC_v3.2.0_win64.zip` を入手して展開し、`adaptFVC.exe` をダブルクリックします（インストールは不要。64 bit 版の Windows 用で、Windows 11 で動作を確認）。初回に「保護されました」と出たら、「詳細情報」→「実行」を押します（署名の無い配布アプリの警告）。
2. 起動すると開く画面で、解析したい画像が入った**フォルダ**を選びます。読み込むのはフォルダの直下にある画像だけです。別のフォルダに切り替えるときは、メニュー「ファイル」→「別のフォルダを開く…」を使います。
3. 長辺が 256 px を超える画像があると、**縮小サイズ**（長辺 1024 / 512 / 256 px）を聞かれます。以後の表示・学習・計算はすべてその縮小版に対して行われ、元の画像は変更されません（選んだサイズより小さい画像は、そのまま使われます）。選択はフォルダごとに記憶されます。選び直すときは、メニュー「設定」→「縮小サイズを選び直す…」のあと、アプリを起動し直して同じフォルダを開き、学習し直します。

選択肢	使いどころ	植被率の差の目安*
1024 px（推奨）	精度優先。通常はこれ	約 +0.2pt
512 px	速度と精度のバランス。処理が遅いとき	約 +0.7pt
256 px	速度優先。**幼苗・細い葉では非推奨**	1pt 超になりうる

*合成画像で、縮小しない場合との植被率の差を測った値（pt はパーセントポイント。+ は過大）。細い葉の幅が 5 px を切ると過大側にずれ始める。同じ試験の中では縮小サイズを揃えることが重要。

4. アプリが、明るさの分布をまんべんなくカバーする画像を選び、右端の列に並べます。この列を**学習用の画像**と呼びます（画像の総数の約1割で、10〜50 枚。総数が 10 枚以下なら全部）。このうち学習に使われるのは、教師点を打った画像だけです。前回の教師点があれば、復元するか聞かれます（第3章）。

2.3 教師点を打つ

1. `Q` を押すと 🌱 **植物**、`W` を押すと 🟫 **土壌**のアノテーションになります（何度押しても同じクラスのまま）。画像のすぐ上の帯と画像の枠がクラスの色（緑＝植物、茶＝土壌）になり、『いま「土壌」をアノテーション中』のように表示されます。

2. 画像上でそのクラスの画素をクリックします。どこに何クリック打つかは 2.4節を参照してください。
3. **連続注釈**（初期設定は ON）：同じクラスで続けてクリックすると、直前のクリック位置から今の位置まで、直線上の画素も自動で教師点になります。5 クリックで線が区切られ、次のクリックから新しい線が始まります（クラスや画像を切り替えたときも同じ）。葉の長軸や土の帯を「なぞる」と効率的ですが、**線がクラスの境界をまたぐと誤った点がまとめて入る**ので、細い葉では注意してください。`X` で OFF にできます。
4. 打ち間違えたら `Z`。クリック単位で戻るので、補間で入った線分もまとめて消えます。
5. `A` / `D` で画像を切り替え、複数の画像に分散して打ちます。ホイールでズーム、中ボタン（右ボタン）でドラッグすると移動します。

2.4 教師点の打ち方のコツと目安

- **多様に打つ。**明るい葉・暗い葉、乾いた土・湿った土・**影。**同じ場所に固めても情報は増えません。
- **葉と土の「芯」に打ち、境界ぎりぎりは避ける。**境界の画素は葉と土の色が混ざった中間色で、人間にもどちらとも言えません。そこに教師点を足して境界を詰めると、ほぼ同じ色に逆のラベルが付き、判定がごま塩状に荒れるだけで、分類モデルの精度（植被率の算出精度）は上がりません。
- **やみくもに増やさず、うまく分類できていない場所に足す。**影が「植物」と判定されたら、影の中の土を「土壌」として打ちます（影の中の葉は「植物」）。植物の無い画像に植被率が付くなら、湿った土・枯れ葉・石など、土のバリエーションを「土壌」として打ちます。

左上の欄に、クラスごとの**クリック数**と、連続注釈で登録された**画素数**が表示されます（例: 「植物: 24 クリック（170 画素）」）。目安はクリック数で示します。

状況	画像の枚数	1枚あたり（各クラス）	合計（各クラス）
まず動かしてみる	3〜5 枚	5 クリック	15〜25 クリック
標準（晴天オルソなど）	10 枚前後	5 クリック	約 50 クリック
条件が複雑（影・水面・枯れ葉）	必要なだけ	—	100 クリック程度まで。それでも直らなければフォルダを分ける（2.1節）

プレビュー（2.5節）で誤分類が見つからなくなったら十分です。登録される画素数はクリック数の数倍〜数十倍になります（正常）。学習には各クラス 10 画素以上が必要です。

2.5 学習と確認

基本は**「学習データの準備 → 学習 → 結果を目で確認 → うまく分類できていない箇所を中心に学習データを追加 → 再学習」**の繰り返しです。通常は2〜3周で誤分類が目立たなくなります。

1. **「学習」**ボタン（`T`）を押します。数秒で終わり、**判定結果（赤＝植物）が画像に重なります。**

2. `A` / `D` で画像を切り替えると、次の画像も自動で判定されます。`Space` で表示を OFF にすると素の画像に戻り、どこに教師点を足すべきか見比べられます。判定が重なったままクリックして追加することもできます。
3. うまく分類できていない場所に正しいクラスの教師点を追加して、再度 `T`。納得できるまで繰り返します。

機能	対象	確かめられること
推論表示（`Space`）	表示中の画像（学習用の画像の列を A / D で切り替える）	教師点を打った画像なら**当てはまり**（教師点のとおりに塗れているか）、打っていない画像なら**汎化**
プレビュー（ボタン）	**学習用の画像の列に入っていない画像**から無作為に最大 10 枚（ボタンの表記は「未注釈の画像」）	**汎化**: 学習に用いていない画像でも正しいか

教師点を打った画像で綺麗に塗れるのは当然です。**本当の確認はプレビューで行います。**「プレビュー」ボタンを押すと、別のウィンドウに判定画像が並びます。うまく分類できていない箇所が目立つ画像をクリックして選び（黄色の枠になる）、「学習用の画像に追加」を押すと、その画像が学習用の画像の列の一番下に加わり、教師点を打てるようになります。画像の総数が 10 枚以下のフォルダでは、全部が学習用の画像に入るのでプレビューは使えません。教師点を打っていない画像を推論表示で確かめてください。**推論表示とプレビューの両方で判定が良好になってから**、全画像の処理へ進みます。

フォルダ内の好きな画像を学習用の画像の列に加えるときは、左パネルの「画像を追加...」を使います。列から外すときは、サムネイルを右クリックして「学習用の画像から外す」を選びます（その画像に打った教師点は、学習に使われ続けます）。

2.6 全画像の処理

「**全画像を処理**」ボタンを押すと、フォルダ内の全画像を並列処理します（数千枚でも数分）。結果は `adaptFVC/results/run_<日時>/` に保存されます（中身は第3章）。学習のあとに教師点を足したり消したりしていると、処理の前に学習し直すかを聞かれます。「はい」を選ぶと、いまの教師点で学習し直してから処理します。

2.7 透明な部分（プロットの外）と GIS との連携

QGIS でオルソ画像から斜めのプロットを切り出すとき、nodata（データの無い領域）を透明にする設定で PNG または TIFF に書き出すと、四隅のプロットの外は透明になります。adaptFVC は、アルファ（透明度）が 128 未満の画素を透明とみなし、「存在しない画素」として扱います。

- 植被率の**分母にも分子にも入れません**（含めると、実際より低い値になります）。
- 透明な画素に打った教師点は、学習から除外されます。
- 判定画像では**グレー**で塗り、Excel の「有効画素率」列に有効な画素の割合を出します。100% なら、画像に透明な部分はありません。

> **圃場全体を地図で見たいとき:** 切り出すときにファイル名をプロット ID にし、Excel を QGIS でプロットのポリゴンに属性結合してください（Excel の「画像ファイル名」列は、縮小した場合も元の画像のファイル名です）。adaptFVC 自身は座標系を扱いません。画像のフォルダから植被率の表を作る道具、と役割を限っています。

3. 出力と自動保存

出力はすべて、画像フォルダの中に新しく作られる `adaptFVC` というフォルダに保存されます。元の画像は変更しません（このフォルダを消しても、元の画像は消えません）。**保存はすべて自動で、保存の操作は要りません。**

```
画像フォルダ/adaptFVC/
    cache/                  縮小版キャッシュ
    models/                 model_latest.pkl / annotations_latest.json / history/
    results/run_<日時>/      全画像の処理1回分（下の表）と overlays/（判定画像）
    previews/               プレビュー（使い捨て）
    logs/                   作業ログ（送信はされない）
```

いつ	何が保存されるか
教師点を打つ・消すたび	`models/annotations_latest.json`
学習するたび	`models/model_latest.pkl` と、教師点の履歴（`models/history/`）
全画像を処理するたび	`results/run_<日時>/` 一式（下の表）

results/run_<日時>/ のファイル	中身
`vegetation_coverage_<日時>.xlsx`	「結果」シートに画像ごとの植被率と有効画素率（「画像ファイル名」列は元の画像の名前）、「モデル情報」シートに学習の設定と、アプリ・ライブラリの版
`annotations_<日時>.json`	この処理に使ったモデルの教師点（人が読める形式）。ここからモデルを作り直せる（第10章）
`model_<日時>.pkl`	学習済みモデル。教師点・学習の設定・アプリと主なライブラリの版を同梱
`overlays/`	判定画像（判定結果を元の画像に半透明で重ねた画像。透明な部分はグレー）
`run_metadata.json`	アプリ・Python・scikit-learn・numpy・OpenCV の版と OS、学習の設定（Random Forest、200 本、シード値 42、色空間 Lab）、縮小サイズ、画像数、処理日時

作業の再開と受け渡し: 同じフォルダを開き直すと、前回の教師点を復元するか聞かれます（学習のあとに教師点を変えていた場合は、その場で学習し直します）。作業を別の PC で再開するときや、共同作業者に渡すときは、「2 学習と確認」の「**別名で保存**」（モデルの .pkl と教師点の .json を並べて保存）と「**モデル/教師点を読み込む**」を使います。読み込むときは、いまの教師点を置き換えるか、追加するかを聞かれ

ます。教師点は画像の**ファイル名**で対応付けられ、教師点の .json だけを読み込んだ場合は、その場で学習します。

4. 結果を信じる前に — 計測の作法

> **数字だけを見て報告してはいけません。**以下は計測の作法です。

1. **判定画像を必ず目視してください。**撮影日（フォルダ）ごとに、明るい画像と暗い画像から数枚ずつ選び、影が植物に、白飛びした葉が土壌になっていないかを確かめます。判定画像は `results/run_<日時>/overlays/` にあります（「出力フォルダを開く」ボタン）。
2. **モデルは撮影条件ごとに作ってください**（2.1節）。
3. **おかしな値は、まず測り方を疑ってください。**植被率が前の日付や隣のプロットと比べて急に跳ね上がったり落ち込んだりしたら、発見と考える前に、教師点の誤りや撮影条件の混入を疑います。

5. トラブルシューティング

症状	対処
影が「植物」になる	影の中の土を「土壌」として打ち、再学習（2.4節）
暗い画像だけ精度が悪い	暗い画像にも教師点を打つ。改善しなければフォルダを分ける（2.1節）
植物ゼロの画像に植被率が付く	土のバリエーションを「土壌」として打つ（2.4節）
プロットの外が計算に入っている気がする	Excel の「有効画素率」を確認。100% なら透明部分が無いので、切り出し時に nodata を透明にする設定を確認（2.7節）
「各クラスに 10 画素以上の教師点が必要です」と出る	どちらかのクラスの教師点が 10 画素に足りない。透明な画素に打った点は除外される（2.7節）
「教師点から学習し直しました」と出る	正常。scikit-learn の版が違う環境で保存されたモデルを、同梱の教師点から作り直している（第10章）
前回の教師点を復元したい	同じフォルダを開けば聞かれる（第3章）。復元を断ったあとに教師点を打つと、自動保存の教師点は上書きされる。そのときは `models/history/` にある学習ごとの記録を「モデル/教師点を読み込む」で選ぶ
最初からやり直したい	メニュー「設定」→「リセット…」。教師点を削除すると、自動保存の教師点も空になる（学習ごとの記録は `models/history/` に残る）
Windows で「保護されました」	「詳細情報」→「実行」（2.2節）
処理が遅い	縮小サイズを小さくする（2.2節。ただし同じ試験の中では揃える）。ノート PC は電源につなぐ

問い合わせるとき: 作業ログは `adaptFVC/logs/` に自動で残ります（送信はされません）。作成者にアプリの使用感を相談いただく場合や、不具合を報告いただく場合は、このログを添えていただけると助かります。ログには、画像フォルダの名前と画像のファイル名が含まれます。

6. ショートカット一覧

キー	動作
クリック	教師点を打つ（連続注釈が ON なら、同じクラスで続けると直線補間）
`Q`	植物をアノテーション
`W`	土壌をアノテーション
`Z`	表示中の画像で、直前のクリックを取り消し
`X`	連続注釈 ON/OFF
`T`	学習
`Space`	推論結果の表示 ON/OFF（ON のあいだは画像切替で自動推論）
`A` / `D`（← / →）	前の画像 / 次の画像
ホイール	ズーム。中ボタン（右ボタン）ドラッグで移動

第2部　理解する・記録する

植被率と判定の原理、設計の考え方、再現性、論文での引用のしかた。方法を人に説明するときや、論文の方法の節を書くときに読んでください。

7. 植被率（FVC）とは何か

> **植被率（FVC: Fractional Vegetation Cover）** ＝ 真上から見たとき、**地面のうち緑の植物に覆われている面積の割合**。0%（裸地）〜100%（葉で地面が隠れる）。

作物が育つほど葉が広がり地面を覆うので、植被率は**「どれだけ育ったか」を刈り取らずに数字にできる指標**です。同じ場所を毎週撮れば成長の速さが、品種ごとに比べれば生育差が分かります。また作物学では、群落を上から撮影した写真から植被率を求め、群落が日射を受け止める割合（受光率）の代わりに用いることもあります（白岩ら 2011）。

ただし植被率は「上から見える緑の割合」であって植物の量そのものではありません。葉が重なるようになると、量が増えても 100% で頭打ちになります。**どんな指標にも測れる範囲がある**ことを覚えておいてください。

8. 原理 — どうやって植物の画素を見分けるか

8.1 画像は数値の表、見分けるのは分類問題

入力は普通のカラー写真（RGB 画像）です。画像は**ピクセル（画素）**の集まりで、1画素は **R・G・B の3つの数値**（各 0〜255）を持ちます。

$$\text{植被率 [\%]} = (\text{「植物」と判定された画素数}) \div (\text{画像の有効な画素数}) \times 100$$

「有効な」とは、画像の一部が透明（プロットの外など）なら、そこを分母にも分子にも入れないという意味です（2.7節）。普通の写真では全画素が有効です。

つまり本当の問題は割り算ではなく、各画素が植物か土壌かを見分けることです。画素が持っているのは色の数値だけで、どれが「植物」でどれが「土壌」かは、そのままでは決まりません。色の数値から「植物」か「土壌」かに振り分けることを**2クラス分類**と呼び、adaptFVC はこれを全画素に対して行います。ここでいう「土壌」は、緑の植物以外のすべて（土・石・枯れ葉・影など）を指します。

8.2 色空間 — RGB を「分けやすい座標」に変換する

adaptFVC は RGB を **CIELAB（Lab）色空間**に変換してから分類します。

成分	意味
L*	明るさ（0=黒 〜 100=白）
a*	緑 ↔ 赤（小さいほど緑。0 が無彩色）
b*	青 ↔ 黄（大きいほど黄。0 が無彩色）

晴れと曇りで明るさ（L*）は大きく変わりますが、葉の「緑さ」（a*）は比較的保たれます。**分けたい性質が特定の軸に集まる座標系**を選ぶと分類は簡単になります。色空間は Lab に固定してあり、誰が使っても同じ条件で計算されます。

8.3 教師あり学習 — 人が見本を示し、機械が境界を学ぶ

Lab のどこからが「植物」かは画像セットごとに違うので、**その画像セットの中で人が見本を示します**。植物の画素と土壌の画素をクリックして「この画素は植物」「この画素は土壌」というラベルを付ける作業が**アノテーション**（学習データの作成）で、ラベルを付けた画素を**教師点**と呼びます。1回のクリックで登録される教師点は1画素以上です（連続注釈では線上の画素すべて。2.3節）。本書では、操作の量を**クリック数**、学習に使われる量を**画素数**で数えます。

アプリは教師点から **Random Forest** で、植物と土壌を分ける**境界**を学習します。Random Forest は、「明るさ L* が 49 以下か？」「a* が −20 以下か？」のような単純な質問を枝分かれ状に重ねて答えを出す**決定木**を 200 本作り、その多数決で分類します（図3）。1本1本の木は粗くても、多数決にすると判定が安定します。こうして色空間の中に境界ができれば（図4）、あとは全画素を機械的に振り分けるだけです。学習したモデルで画素を植物と土壌に振り分けることを**推論**と呼びます。**人が少数の見本を与え、機械がルールを学び、大量のデータに適用する**。これが教師あり学習の最も基本的な形です。

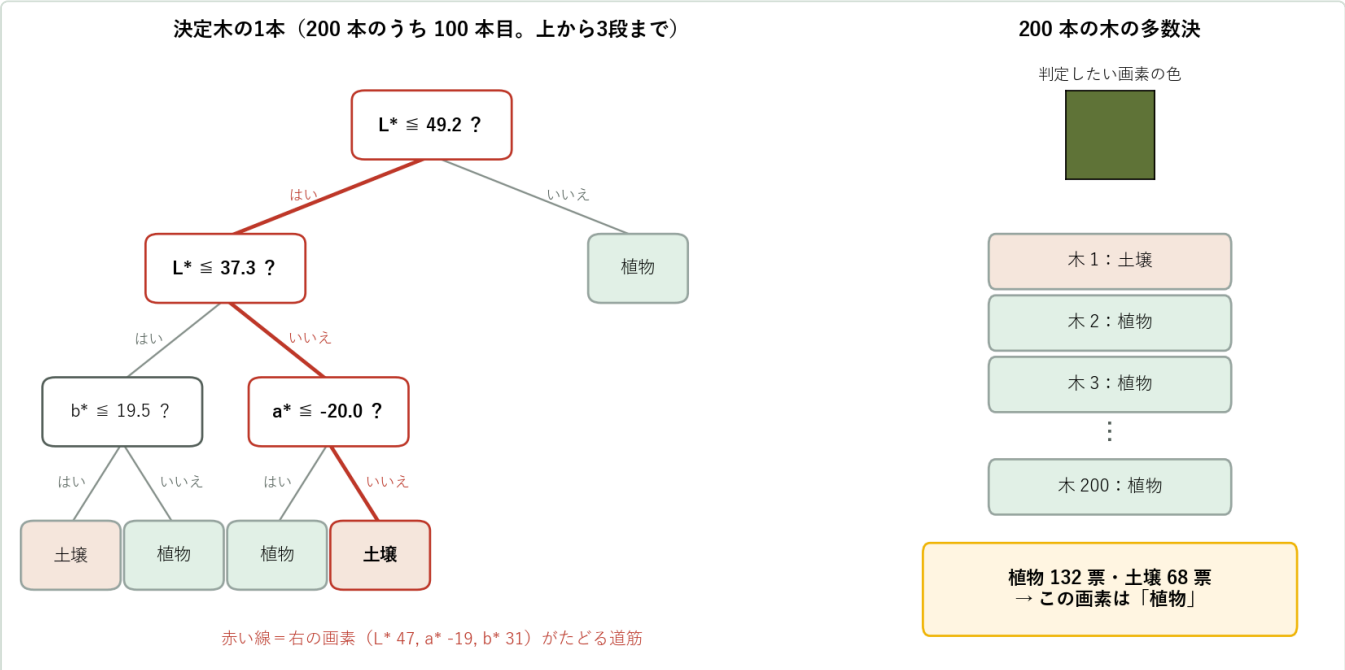


図3｜Random Forest の判定のしかた（図1の教師点で実際に学習したモデル）。左: 200 本の決定木のうち1本の、上から3段まで（質問に使う成分としきい値は、木ごとに異なる）。画素の色についての「はい／いいえ」の質問をたどり、行き着いた先が判定になる（「…」の先にも質問が続く）。赤い線は右の画素がたどる道筋。右: 同じ画素を 200 本すべてに通すと、植物 132 票・土壌 68 票で「植物」と判定される。この画素は葉と土壌の中間の色なので、木によって答えが割れる。

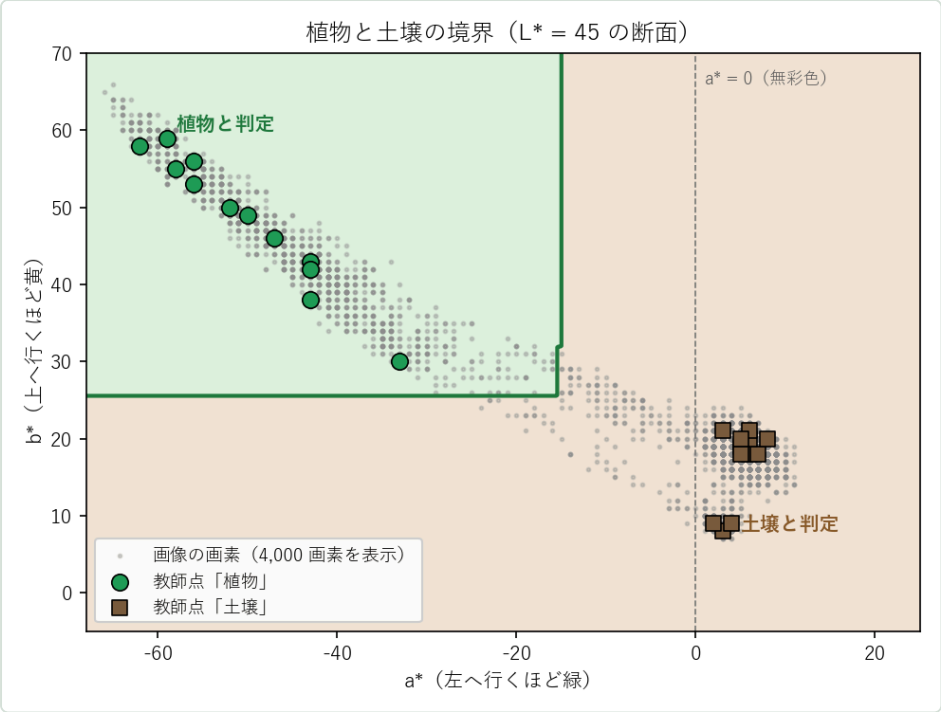


図4｜植物と土壌を分ける境界を色空間のどこに置くかを、機械が自ら学習する。図1の画像の画素を Lab の a*-b* 平面に置いたもの（灰色。4,000 画素を間引いて表示）。教師点（緑丸=植物、茶四角=土壌）から Random Forest が学習した境界を緑の線で示す（L* を画像の中央値 45 に固定した断面）。決定木の質問が「〜以下か」の形なので、境界は軸に沿った階段状になる。

9. 設計の考え方

9.1 全自動のしきい値法との違い

見本からの学習を経ずに境界を全自動で決める方法の代表が、植生指数 ExG を使った Otsu の二値化（**ExG + Otsu**）です。ExG（2G − R − B）で「緑の強さ」を1つの数値にし、Otsu 法でしきい値を画像ごとに自動で決めて2つに割ります。手軽で多くの場面で動きますが、ルールが単純なぶん、次の2つの限界があります。

- **見ている情報が少ない。**判断の材料は ExG という1つの数値だけです。強い影、枯れ葉、水面の映り込みのように、ExG では植物と土壌の差が小さくなるものを分けられず、精度が落ちます。
- **必ず2つに割る。**Otsu 法は「画像には2つのグループがある」と仮定し、その画像の中での相対的な分け目を引きます。植物がどう見えるかは知らないので、植物が写っていない画像でも土の色むらを「植物」と「土壌」に割り、存在しない植生を作り出します（図5）。

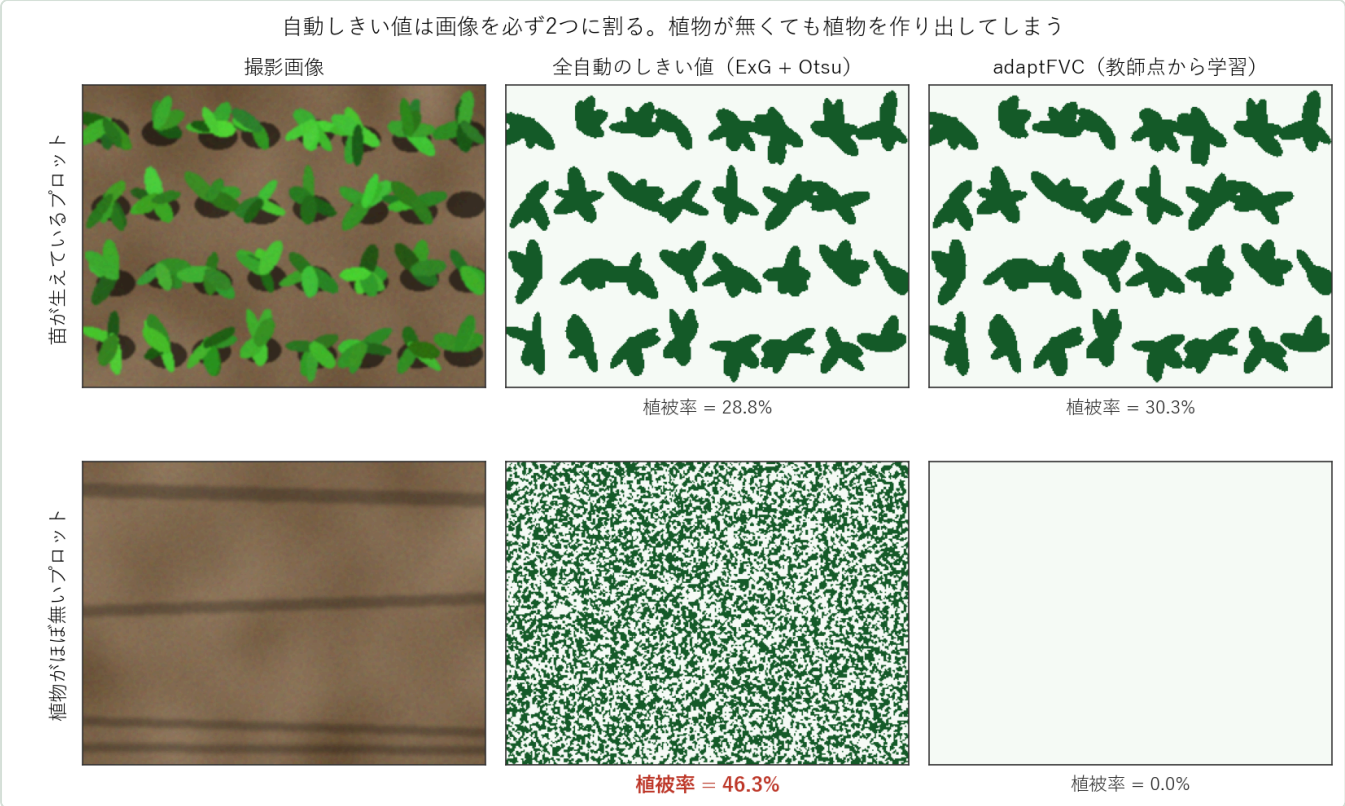


図5｜全自動のしきい値法が破綻する例。上段: 苗が写っているプロットでは、ExG+Otsu も adaptFVC もほぼ同じ（植被率 28.8% と 30.3%）。下段: 植物がほぼ無いプロット（播種前・欠株）で、ExG+Otsu は土の色むらを2つに割って**植被率 46.3% という存在しない植生**を報告した。上段と同じ教師点で学習したモデルは正しく 0.0%。濃い緑が「植物」と判定された画素（図1・図2の赤い重ね表示とは表し方が異なる）。

adaptFVC は、Lab の3つの値を使い、人が示した見本から「この画像セットでは植物と土壌はこう見える」を学びます。判断の材料が多く、植物の見た目を見本として知っているので、この2つの限界を補えます。撮影日・圃場・カメラ・生育段階が変われば「緑」の見え方も変わるので、モデルは画像セットごとに作ります（2.1節）。見本に合わせてモデルを画像セットに**適応（adapt）**させる。これが名前の由来です。また、最後は必ず人の目で結果を確かめる（第4章）ので、学習と確認を人が繰り返す前提で設計しています（2.5節）。

> **覚えてほしい一文:** 全自動の方法は、ルールが単純なぶん、それに合わない画像では黙って間違える。adaptFVC は、人が示した見本から画像セットごとにルールを学ぶことで、その限界を補う。

9.2 深層学習を使わない理由

機械学習は、データからルールを学ぶ手法の総称です。adaptFVC の Random Forest は、人が決めた特徴（ここでは Lab の3つの値）からルールを学ぶ、古典的な機械学習の手法です。**深層学習**は機械学習の一種で、多層のニューラルネットワークを使い、色だけでなく葉の形や模様といった特徴も画像から自ら学びます。表現力が高い一方で、次の理由から adaptFVC では採用していません。

- **学習データの準備に大きなコストがかかる。**特徴まで画像から学ばせるには、画素単位で「植物」「土壌」を塗り分けた正解画像を大量に用意する必要があります。

- **汎化が難しい。**あらゆる圃場・カメラ・撮影環境・生育ステージにまたがって使える汎用的なモデルには、あらゆる条件を網羅した学習データが要ります。網羅的な収集とアノテーションは現実的ではなく、学習した条件の外では、モデルをそのまま使い回せません。
- **重い。**学習に GPU と時間が要り、推論の計算量も大きくなります。

そこで adaptFVC は、狭い範囲の画像セットごとに数分で学習データを準備し、軽量なモデルを作ります（2.1節）。**モデルは使い捨てでよい**、という考え方です。これは深層学習が不要という意味ではなく、「計測の道具として今日確実に動くのは何か」という判断です。

10. 再現性 — 同じモデルを作り直せる

adaptFVC のモデルは教師点から作られます。学習に使う Random Forest は乱数のシード値を固定しているので、**同じ画像に打った同じ教師点からは、縮小サイズ、アプリの版、scikit-learn（adaptFVC が内部で使う機械学習のライブラリ）の版がすべて同じなら、何度学習しても同じモデル**ができます。つまり、教師点の JSON ファイル（第3章）と元の画像が残っていれば、この条件のもとでモデルを作り直せます。作り直すには、同じ画像フォルダを開き、「モデル/教師点を読み込む」で教師点の JSON を選びます（その場で学習します）。全画像の処理のたびに結果フォルダへ保存される JSON は、その処理に使ったモデルの教師点です。

2つの結果が同じモデルによるものか確かめるには: 両方の `run_metadata.json` で、アプリの版・scikit-learn の版・学習の設定・縮小サイズが一致し、教師点の JSON と教師点を打った画像が同じであれば、同じモデルです。保存したモデルを「モデル/教師点を読み込む」で読み込んだとき、scikit-learn の版などが違ってそのまま使えなければ、アプリが同梱の教師点から自動で学習し直し、そのことを表示します。

11. 論文などで引用するとき

adaptFVC で計算した植被率を論文などで使うときは、ソフトウェアとして引用してください。GitHub のリポジトリ（github.com/KotaNakajima/adaptFVC）のページ右側にある「Cite this repository」から、APA 形式や BibTeX 形式の引用文をコピーできます。リリースごとに付く DOI は、取得後にこの章とリポジトリに記載します。

方法の節には、少なくとも次を書くと、他の人が同じ計算をたどれます。

- アプリの版（例: adaptFVC v3.2.0）
- 学習器と特徴量（Random Forest、決定木 200 本、乱数のシード値 42、Lab 色空間。いずれも固定仕様）と、scikit-learn の版（`run_metadata.json` に記録）
- 縮小サイズ（長辺 px）
- モデルを作った単位（例: 撮影日ごと）
- 透明な画素（アルファ 128 未満）を除外したこと（該当する場合）

結果を再現できるように、教師点のファイル（結果フォルダの `annotations_<日時>.json`）を補足資料として公開してください。このファイルから同じモデルを作り直せます（第10章）。

書き方の例（数値は例）：「植被率は adaptFVC v3.2.0 で算出した。画像を長辺 1024 px に縮小し、撮影日ごとに植物と土壌の教師点を与えてモデルを作った（教師点のファイルは補足資料 S1）。各画素の CIELAB 値を特徴量とし、Random Forest（Breiman 2001。決定木 200 本、乱数のシード値 42、scikit-learn 1.8.0）で全画素を植物と土壌に分類した。」比較に ExG（Woebbecke ら 1995）と Otsu 法（Otsu 1979）を使う場合の原典も、参考文献に挙げました。

12. ミニ用語集

用語	意味
植被率（FVC）	真上から見た画像で、緑の植物が覆う面積の割合（%）
ピクセル（画素）	画像を構成する最小の点。RGB の3つの数値を持つ
アルファ（透明度）	画素の4つ目の数値で、不透明度を表す（0＝完全に透明、255＝不透明）。adaptFVC はアルファが 128 未満の画素をプロットの外とみなし、計算から除く
色空間	色を数値の組で表す座標系（RGB、Lab など）
アノテーション	画素に「植物」「土壌」の正解ラベルを付ける作業。学習データの作成にあたる
教師点	アノテーションでラベルを付けた画素。モデルはこれから学習する
機械学習	データからルールを学ぶ手法の総称
教師あり学習	機械学習の手法の区分のひとつ。人が与えた正解例からルールを学ぶ
深層学習	機械学習の一種。多層のニューラルネットワークで、色だけでなく形や模様などの特徴も画像から学ぶ
決定木	「〜以下か？」という質問を枝分かれ状に重ねて答えを出す仕組み（図3）
Random Forest	多数の決定木の多数決で分類する手法。木を作るときの乱数をシード値で固定しているので、同じ教師点からは何度学習しても同じモデルができる（条件は第10章）
推論	学習したモデルで、画素を植物と土壌に振り分けること
乱数のシード値	乱数の出方を決める初期値。同じ値なら毎回同じ乱数の列になる
Otsu 法	画素の値（adaptFVC の比較では ExG の値）のヒストグラムから、2つのグループの分かれ目になるしきい値を自動で決める方法。必ず2つに割る
判定画像	判定結果を元の画像に半透明で重ねた確認用の画像（結果フォルダの `overlays/`）
プロット	試験区。圃場の中で区切った1区画
セグメンテーション	画像を画素単位で領域（ここでは植物と土壌）に分けること

参考文献

Breiman, L. (2001) Random forests. Machine Learning 45(1): 5–32. https://doi.org/10.1023/A:1010933404324

Otsu, N. (1979) A threshold selection method from gray-level histograms. IEEE Transactions on Systems, Man, and Cybernetics 9(1): 62–66. https://doi.org/10.1109/TSMC.1979.4310076

白岩立彦・川崎洋平・本間香貴（2011）作物群落の日射利用効率の推定．日本作物学会紀事 80(3): 360–364. https://doi.org/10.1626/jcs.80.360

Woebbecke, D. M., Meyer, G. E., Von Bargen, K. and Mortensen, D. A. (1995) Color indices for weed identification under various soil, residue, and lighting conditions. Transactions of the ASAE 38(1): 259–269. https://doi.org/10.13031/2013.27838

---

adaptFVC v3.2.0 取扱説明書（2026-11-10）／ソースコード: github.com/KotaNakajima/adaptFVC ／ 図1・図3〜5は、アプリと同じ計算コードで合成画像を実際に計算したもの（docs/make_figures.py で作り直せる）。図2は docs/make_screenshot.py で撮影。